



AI휴먼 과정

9일차

Python 전처리 및 시각화 - NumPy 배열과 벡터 연산

오늘의 학습 목표

- 1 NumPy가 필요한 이유를 Python 리스트와 비교해 설명할 수 있습니다.
- 2 ndarray의 차원, 크기, 형태(shape), 자료형(dtype)을 확인할 수 있습니다.
- 3 1차원·2차원 배열을 생성하고 원하는 위치의 값을 조회할 수 있습니다.
- 4 슬라이싱과 조건 인덱싱으로 필요한 데이터만 추출할 수 있습니다.
- 5 배열 간 산술 연산과 스칼라 연산을 벡터 연산으로 수행할 수 있습니다.
- 6 axis를 사용해 행·열 방향의 합계와 평균을 계산할 수 있습니다.
- 7 배열 형태를 reshape()로 변경하고 분석에 필요한 구조로 변환할 수 있습니다.

Numpy : 데이터 분석의 출발점

Python 기본 문법에서는 하나의 값 또는 여러 값을 리스트·딕셔너리로 다뤘습니다.
데이터 분석에서는 수백 개에서 수백만 개의 숫자를 반복적으로 계산해야 합니다.

예시

- 학생 1,000명의 성적 평균 계산
- 센서 10만 건의 측정값 변환
- 이미지 픽셀의 밝기 조절
- 머신러닝 입력 데이터의 수치 변환

이런 작업에서 NumPy는 많은 숫자를 배열 형태로 저장하고 빠르게 계산하는 핵심 도구입니다.

흐름

Python 문법 → NumPy 배열 → Pandas DataFrame → 시각화 → 머신러닝/딥러닝

NumPy란?

NumPy = Numerical Python

- Python에서 수치 계산을 효율적으로 수행하기 위한 라이브러리
- 핵심 자료구조는 ndarray
- 다차원 배열을 지원
- 반복문을 직접 작성하지 않고 배열 전체에 연산 가능
- Pandas, Matplotlib, scikit-learn, TensorFlow, PyTorch 등 데이터·AI 라이브러리의 기반

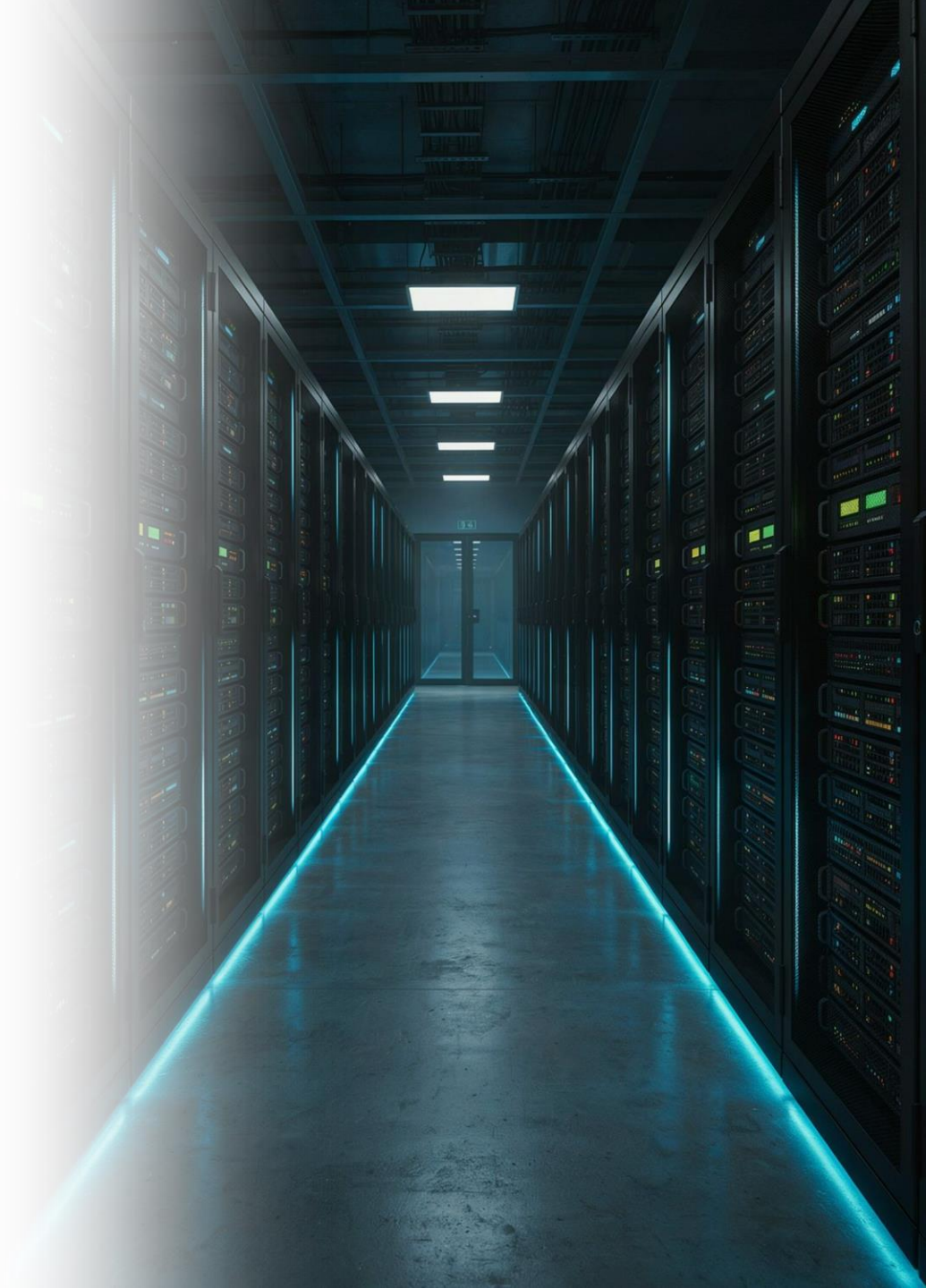
설치

```
pip install numpy
```

불러오기

```
import numpy as np
```

관례적으로 numpy를 np라는 별칭(alias)으로 사용합니다.



핵심 용어 정리

용어	영문/표기	설명
NumPy	Numerical Python	수치 계산을 위한 Python 라이브러리
배열	Array	같은 성격의 데이터를 순서대로 저장한 구조
다차원 배열	ndarray	NumPy의 핵심 배열 객체
차원	Dimension	배열을 표현하기 위해 필요한 축의 개수
축	Axis	배열을 계산하거나 집계하는 방향
형태	shape	각 축에 몇 개의 원소가 있는지 나타내는 튜플
차원 수	ndim	배열이 몇 차원인지 나타내는 정수
자료형	dtype	배열 내부 원소의 데이터 타입
원소 개수	size	배열에 포함된 전체 원소 수
벡터 연산	Vectorized Operation	반복문 없이 배열 전체에 한 번에 수행하는 연산
브로드캐스팅	Broadcasting	서로 다른 shape의 배열을 일정 규칙에 따라 연산 가능하게 확장하는 기능

Python 리스트와 NumPy 배열 비교

Python 리스트

```
scores = [70, 85, 90, 95]
result = []
for score in scores:
    result.append(score + 5)
print(result)
```

NumPy 배열

```
import numpy as np
scores = np.array([70, 85, 90, 95])
print(scores + 5)
```

핵심 차이

- 리스트의 +는 연결(concatenation)에 사용될 수 있음
- NumPy 배열의 +는 원소별 산술 연산
- NumPy는 동일 자료형 중심으로 메모리를 연속적으로 관리하여 수치 계산에 적합

ndarray 생성 - np.array()

가장 기본적인 배열 생성 방법

```
import numpy as np
scores = np.array([70, 85, 90, 95])
print(scores)
print(type(scores))
```

출력 예

```
[70 85 90 95]
<class 'numpy.ndarray'>
```

2차원 배열

```
scores2 = np.array([
    [80, 90, 85],
    [70, 75, 95],
    [88, 92, 90]
])
```

배열 생성 주요 함수

함수	용도	예
<code>np.array()</code>	리스트·튜플 등을 배열로 변환	<code>np.array([1, 2, 3])</code>
<code>np.arange()</code>	일정 간격의 연속 값 생성	<code>np.arange(0, 10, 2)</code>
<code>np.linspace()</code>	구간을 동일 간격으로 나눔	<code>np.linspace(0, 1, 5)</code>
<code>np.zeros()</code>	0으로 채운 배열 생성	<code>np.zeros((2, 3))</code>
<code>np.ones()</code>	1로 채운 배열 생성	<code>np.ones((2, 3))</code>
<code>np.full()</code>	지정 값으로 채운 배열 생성	<code>np.full((2, 2), 7)</code>
<code>np.eye()</code>	단위행렬 생성	<code>np.eye(3)</code>

배열의 구조 확인 - shape, ndim, size, dtype

```
import numpy as np
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
print(arr.shape)
print(arr.ndim)
print(arr.size)
print(arr.dtype)
```

출력 예

```
(2, 3)
2
6
int64
```

해석

- shape (2, 3) : 2행 3열
- ndim 2 : 2차원
- size 6 : 전체 원소 6개
- dtype : 배열 내부 원소의 자료형

주요 dtype

dtype 계열	의미	예
int8, int16, int32, int64	정수	점수, 개수
uint8 등	부호 없는 정수	이미지 픽셀 0~255
float32, float64	실수	평균, 센서값
bool	참/거짓	조건 결과
str_	문자열	문자 데이터

자료형 지정

```
arr = np.array([1, 2, 3], dtype=np.float64)
print(arr)
print(arr.dtype)
```

자료형 변환

```
converted = arr.astype(np.int32)
```

NumPy 주요 상수

상수	의미	예
np.pi	원주율 π	np.pi
np.e	자연상수 e	np.e
np.inf	무한대	np.inf
np.nan	숫자가 아닌 값/결측 표현	np.nan
np.newaxis	새 축을 추가할 때 사용	arr[:, np.newaxis]

샘플

```
import numpy as np
print(np.pi)
print(np.e)
print(np.inf)
print(np.nan)
```

⚠ 참고

np.nan == np.nan은 False입니다.
결측 여부 확인에는 np.isnan()을 사용합니다.

1차원 배열 인덱싱

```
arr = np.array([10, 20, 30, 40, 50])  
print(arr[0])  
print(arr[2])  
print(arr[-1])
```

값 변경

```
arr[1] = 99  
print(arr)
```

핵심

- 인덱스는 0부터 시작
- 음수 인덱스로 뒤에서 접근 가능
- Python 리스트의 인덱싱 규칙과 유사

2차원 배열 인덱싱

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
print(arr[0, 0])
print(arr[1, 2])
print(arr[2, 1])
```

의미

arr[행 인덱스, 열 인덱스]

행 전체

```
print(arr[1])
```

특정 열

```
print(arr[:, 1])
```

슬라이싱

1차원

```
arr = np.array([10, 20, 30, 40, 50, 60])  
print(arr[1:4])  
print(arr[:3])  
print(arr[::-2])
```

2차원

```
matrix = np.array([  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [9, 10, 11, 12]  
)  
print(matrix[0:2, 1:3])
```

📌 슬라이싱 구조: 배열[시작:끝:간격] — 끝 인덱스는 포함되지 않습니다.

조건 인덱싱(Boolean Indexing)

조건에 맞는 데이터만 추출할 수 있습니다.

```
scores = np.array([55, 70, 82, 91, 68, 88])  
condition = scores >= 80  
print(condition)  
print(scores[condition])
```

한 줄로 표현

```
print(scores[scores >= 80])
```

여러 조건

```
print(scores[(scores >= 70) & (scores < 90)])
```

⚠ 주의 NumPy 배열의 복합 조건에서는 `and`, `or` 대신 `&`, `|`를 사용하고 각 조건을 괄호로 묶는 것이 안전합니다.

벡터 연산

```
arr = np.array([10, 20, 30, 40])  
print(arr + 5)  
print(arr - 3)  
print(arr * 2)  
print(arr / 10)  
print(arr ** 2)
```

두 배열 연산

```
a = np.array([1, 2, 3])  
b = np.array([10, 20, 30])  
print(a + b)  
print(a * b)
```

각 위치의 원소끼리 연산됩니다.

벡터화(Vectorization)의 의미

일반 반복문

```
values = [10, 20, 30, 40]
result = []
for value in values:
    result.append(value * 1.1)
```

NumPy 벡터 연산

```
values = np.array([10, 20, 30, 40])
result = values * 1.1
```

장점

- 코드가 짧고 의도가 명확함
- 대량 수치 계산에 적합
- NumPy 내부의 최적화된 연산을 활용

데이터 분석에서 가능한 한 배열 연산을 우선 고려합니다.

집계 함수

numpy 함수 방식

```
scores = np.array([70, 85, 90, 95, 80])  
print(np.sum(scores))  
print(np.mean(scores))  
print(np.max(scores))  
print(np.min(scores))  
print(np.std(scores))
```

ndarray 메서드 방식

```
print(scores.sum())  
print(scores.mean())  
print(scores.max())  
print(scores.min())  
print(scores.std())
```

axis의 개념

2차원 배열

```
scores = np.array([
    [80, 90, 85],
    [70, 75, 95],
    [88, 92, 90]
])
```

전체 평균

```
print(scores.mean())
```

열별 평균

```
print(scores.mean(axis=0))
```

행별 평균

```
print(scores.mean(axis=1))
```

axis의 값

- axis=0 : 각 열의 결과를 만들 → 열별 집계
- axis=1 : 각 행의 결과를 만들 → 행별 집계

reshape() - 배열 형태 변경

```
arr = np.arange(1, 13)
print(arr)
matrix = arr.reshape(3, 4)
print(matrix)
```

원소 개수가 맞아야 합니다.

가능

- 12개 → (3, 4)
- 12개 → (2, 6)
- 12개 → (4, 3)

불가능

- 12개 → (5, 3)

자동 계산

```
print(arr.reshape(3, -1))
```

-1은 NumPy가 해당 축의 크기를 자동 계산하도록 지정합니다.

배열 형태 변경 관련 메소드

메소드/함수	용도	특징
reshape()	shape 변경	원소 수 유지
flatten()	1차원으로 펼침	복사본 생성
ravel()	1차원으로 펼침	가능한 경우 원본 메모리 공유
transpose() / .T	축 순서 변경	2차원에서는 행·열 전환
squeeze()	크기가 1인 축 제거	(1, 3) → (3,) 등
expand_dims()	축 추가	모델 입력 형태 조정 등에 사용

샘플

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])  
print(matrix.T)  
print(matrix.flatten())
```

브로드캐스팅 기초

브로드캐스팅은 shape가 다른 배열끼리도 일정 규칙을 만족하면 연산할 수 있게 해줍니다.

스칼라와 배열

```
arr = np.array([10, 20, 30])
print(arr + 5)
```

2차원 배열과 1차원 배열

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
bonus = np.array([1, 2, 3])
print(matrix + bonus)
```

bonus가 각 행에 적용됩니다.

활용 예

- 각 과목별 보정점수 적용
- 각 센서별 기준값 차감
- 여러 데이터의 단위 변환

22. 자주 사용하는 NumPy 함수·메소드 표

분류	함수/메소드	설명
생성	np.array()	배열 생성
생성	np.arange()	일정 간격의 값 생성
생성	np.linspace()	지정 구간을 균등 분할
생성	np.zeros(), np.ones()	0/1 배열 생성
구조	.shape, .ndim, .size	형태·차원·원소 수 확인
자료형	.dtype, .astype()	자료형 확인·변환
변형	.reshape()	배열 shape 변경
변형	.flatten(), .ravel()	1차원 배열로 변환
변형	.T, np.transpose()	축 순서 변경
집계	.sum(), .mean()	합계·평균
집계	.min(), .max()	최솟값·최댓값
집계	.std(), .var()	표준편차·분산
검색	np.where()	조건에 따라 위치/값 선택
정렬	np.sort(), np.argsort()	값 정렬/정렬 인덱스
결합	np.concatenate()	기준 축 기준 연결
결합	np.vstack(), np.hstack()	세로/가로 결합
중복	np.unique()	고유값 추출
결측	np.isnan()	NaN 여부 확인

샘플 코드 - 성적 배열 분석

```
import numpy as np
scores = np.array([
    [78, 85, 92],
    [88, 90, 76],
    [95, 82, 89],
    [67, 74, 80]
])
print("배열 형태:", scores.shape)
print("전체 평균:", scores.mean())
print("과목별 평균:", scores.mean(axis=0))
print("학생별 평균:", scores.mean(axis=1))
student_avg = scores.mean(axis=1)
print("평균 85점 이상 학생 인덱스:",
      np.where(student_avg >= 85)[0])
```

핵심

1. 2차원 배열 생성
2. shape 확인
3. axis 기반 평균
4. 조건 기반 대상 추출

흔히 발생하는 오류

reshape 원소 수 불일치

```
arr = np.arange(10)
arr.reshape(3, 4) # 오류
```

복합 조건에서 and 사용

```
scores[(scores >= 70) and (scores < 90)] # 오류
```

수정

```
scores[(scores >= 70) & (scores < 90)]
```

shape가 맞지 않는 배열 연산

```
a = np.array([1, 2, 3])
b = np.array([10, 20])
print(a + b) # 브로드캐스팅 불가
```

dtype에 따른 값 변환

```
arr = np.array([1, 2, 3])
arr[0] = 3.7
print(arr) # 정수 배열에서는 값이 정수형으로 저장됨
```

A person is sitting at a desk in a modern office at night, looking at multiple computer monitors. The desk has a keyboard, a mouse, and a notebook. The office has large windows showing a city skyline at night. The person is wearing a dark sweater and is focused on the work.

실무 연결 - NumPy는 어디에 사용되는가?

데이터 전처리

- 결측값 탐지
- 조건 필터링
- 수치 변환
- 정규화/표준화 계산

이미지

- 이미지는 높이 $\times$ 너비 $\times$ 채널 형태의 숫자 배열
- 픽셀값 조절과 색상 변환에 배열 연산 활용

음성

- 파형 데이터를 1차원 수치 배열로 표현
- 음량·구간·특징값 처리에 활용

머신러닝/딥러닝

- 입력 특성과 정답을 배열 또는 텐서로 처리
- 모델의 입력 shape 이해에 NumPy의 차원 개념이 직접 연결

오늘의 정리와 다음 학습 연결

오늘 반드시 설명할 수 있어야 하는 질문

1. NumPy의 핵심 배열 객체는 무엇인가?
2. shape, ndim, size, dtype은 각각 무엇을 의미하는가?
3. 2차원 배열에서 axis=0과 axis=1의 차이는 무엇인가?
4. 조건 인덱싱은 어떤 상황에서 사용하는가?
5. 벡터 연산이 반복문보다 데이터 분석에 유리한 이유는 무엇인가?
6. reshape()를 사용할 때 지켜야 하는 조건은 무엇인가?

다음 학습

- Pandas Series와 DataFrame
- CSV 파일 읽기
- 데이터 구조 확인
- 기초 통계량 확인

NumPy 배열을 이해하면 Pandas와 머신러닝에서
데이터의 shape와 수치 연산을 훨씬 쉽게 이해할 수 있습니다.